

Análisis de Nubes de Puntos LiDAR con Python y PDAL

Confederación Hidrográfica del Guadalquivir — Curso de Teledetección y SIG

Descripción

En esta sesión trabajamos con **nubes de puntos LiDAR** usando Python, comparando dos fuentes de datos reales sobre la zona de **Alcolea del Río (Sevilla)**:

Fuente	Sensor	Fecha	Características
PNOA-LiDAR 3ª cobertura	Avión	2024	11 tiles 1×1 km, clasificados, ETRS89/UTM30N, altura ortométrica
Vuelo dron	DJI Zenmuse L1 + D-RTK 2	2024	~303 M puntos, RTK con base autoposicionada

Datos

Los datos de la práctica se descargan desde el siguiente enlace:

Descargar datos (SACO CSIC)

Contenido del paquete:

```
datos/  
├─ cloud92048a649bb1c353.copc.laz  ← Nube L1 original (sin CRS, altura elipsoidal)  
├─ l1_ortometrica.copc.laz          ← Nube L1 corregida (ETRS89 + altura ortométrica)  
├─ merged_11tiles.copc.laz          ← Mosaico PNOA 2024 (11 tiles fusionados)  
└─ pnoa_recortado.copc.laz          ← PNOA recortado al área del vuelo L1
```

Los archivos COPC (Cloud Optimized Point Cloud) permiten lectura parcial eficiente: podemos explorar el header o leer solo una zona sin cargar los cientos de millones de puntos en RAM.

Entorno de trabajo

Instalación con conda (recomendado)

```
conda create -n lidar python=3.11  
conda activate lidar  
conda install -c conda-forge pdal python-pdal proj-data laspy lazrs-python \\\brasterio geopandas shapely matplotlib numpy jupyter
```

proj-data es imprescindible: instala el geoide oficial español es_ign_egm08-rednap.tif, necesario para la corrección de alturas elipsoidales → ortométricas.

Verificar el geoide

```
projinfo -s EPSG:4979 -t EPSG:25830+5782
```

Debe aparecer +grids=es_ign_egm08-rednap.tif en la primera operación listada.

Notebook

lidar_pdal_l1_vs_pnoa.ipynb

El notebook cubre las siguientes secciones:

0 · Setup y rutas

Imports y centralización de rutas. Solo hay que editar la variable WORKDIR para adaptar a cada máquina.

1 · Mosaico de tiles PNOA con laspy

Los tiles PNOA se distribuyen en celdas de 1×1 km. Aprendemos a fusionarlos en un único archivo usando escritura incremental (laspy.LasWriter) para no saturar la RAM con 113 M de puntos.

2 · Arquitectura de PDAL

PDAL funciona mediante **pipelines JSON**: cadenas de operaciones que procesan la nube en flujo continuo.

READER → FILTER → FILTER → WRITER

Ventajas: - Eficiente en memoria: los puntos fluyen uno a uno - Reproducible: el JSON es documentación ejecutable - Flexible: se combinan operaciones libremente

Tipo	Ejemplos
readers.*	readers.copc, readers.las, readers.e57
filters.*	filters.crop, filters.range, filters.reprojection, filters.hexbin
writers.*	writers.copc, writers.las, writers.gdal

3 · Inspección de nubes de puntos

- **quickinfo**: lee solo el header (instantáneo, sin cargar datos)
 - Número de puntos, bounds XYZ, dimensiones disponibles
- **Metadatos CRS**: comprobamos que el PNOA tiene CRS correcto y el L1 no
- **filters.stats**: estadísticas por clasificación usando muestra de baja resolución

Clasificaciones estándar LAS:

Código	Clase
1	Sin clasificar
2	Suelo
3	Vegetación baja
4	Vegetación media
5	Vegetación alta
6	Edificio
12	Solapamiento

4 · Visualización de perfiles

Con laspy cargamos una muestra aleatoria de 100 000 puntos y generamos tres vistas por fuente: - **Vista en planta** coloreada por altura - **Perfil lateral** coloreado por clasificación - **Histograma de distribución Z**

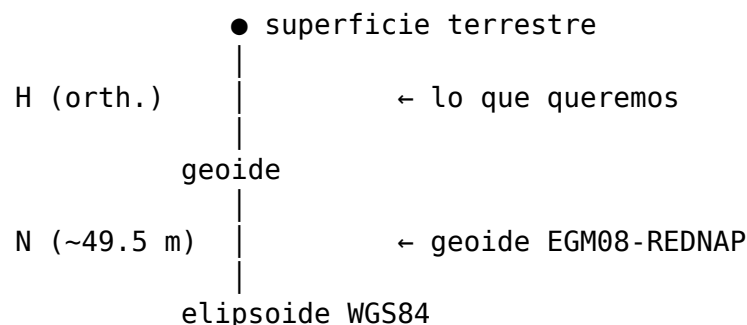
5 · El problema del CRS en DJI Terra

Este es uno de los problemas más frecuentes al trabajar con nubes de puntos de drones DJI.

El problema: DJI Terra exporta con altura **elipsoidal WGS84** y no escribe el CRS en el header del LAS.

Por qué importa: La diferencia entre la altura elipsoidal y la ortométrica (lo que llamamos “altura sobre el nivel del mar”) es de ~49.5 m en la zona de Alcolea del Río.

$H \text{ (ortométrica)} = h \text{ (elipsoidal)} - N \text{ (ondulación del geoide)}$



Solución en PDAL:

```
pipeline = {
  "pipeline": [
    {
      "type": "readers.copc",
      "filename": "l1_original.copc.laz",
      "override_srs": "EPSG:32630+4979" # forzamos el CRS que Terra no escribió
    },
    {
      "type": "filters.reprojection",
      "out_srs": "EPSG:25830+5782" # ETRS89/UTM30N + altura ortométrica
    },
    {
      "type": "writers.copc",
      "filename": "l1_ortometrica.copc.laz"
    }
  ]
}
```

- EPSG:32630+4979 = WGS84/UTM 30N + altura elipsoidal WGS84
- EPSG:25830+5782 = ETRS89/UTM 30N + altura ortométrica (datum Alicante)

Incluimos un **sanity check** rápido con "resolution": 5.0 que tarda segundos en verificar que la corrección es correcta antes de lanzar el proceso completo (~15 min para 303 M puntos).

6 · Recorte del PNOA al área de vuelo

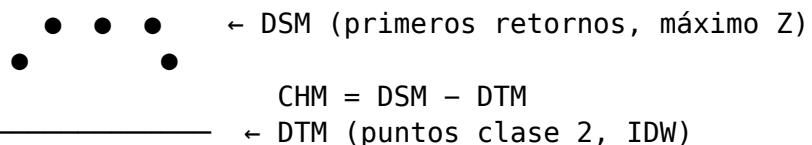
Para comparar ambas fuentes necesitamos recortar el PNOA al mismo extent que el L1.

Estrategia en dos pasos (eficiente):

1. filters.crop con **bounding box** → descarta el 90% del mosaico PNOA en milisegundos
2. filters.hexbin → calcula el **hull real** del vuelo (contorno irregular hexagonal)
3. filters.crop con el polígono WKT del hull → recorte ajustado al área real

El hull se exporta como GeoJSON para visualización en QGIS.

7 · DTM, DSM y CHM



Modelo	Puntos usados	Interpolación
DTM (terreno desnudo)	Solo clase 2 (suelo)	IDW, window_size=5
DSM (superficie superior)	Primeros retornos (ReturnNumber=1)	Máximo Z
CHM (altura de vegetación)	—	DSM – DTM

Los bounds se fijan a valores enteros para alineación **píxel a píxel** entre L1 y PNOA, requisito para la resta y comparación.

Las funciones comprueban si los rasters ya existen antes de procesar (idempotentes).

8 · Comparación visual

Grid 2×3 con los 6 rasters (DTM/DSM/CHM × L1/PNOA) con escalas de color compartidas para comparación directa.

9 · Cuantificación y corrección del sesgo Z

¿Por qué hay un sesgo residual después de corregir el geoide?

La base D-RTK 2 fue usada en **modo autoposicionamiento** (modo A): se posiciona por GNSS autónomo con un error absoluto típico de 2–5 m en vertical.

La precisión **relativa** interna de la nube sigue siendo centimétrica. Solo la **exactitud absoluta** está degradada.

Corrección: 1. Calcular DTM_L1 – DTM_PNOA en toda la zona 2. Calcular la **mediana** (robusta frente a outliers de vegetación) 3. Aplicar el offset con `filters.assign`:

```
{"type": "filters.assign", "value": "Z = Z - <sesgo_mediana>"}
```

Conceptos y herramientas

Herramienta	Uso en esta práctica
pdal	Pipelines de procesamiento de nubes de puntos
laspy	Lectura/escritura LAS y mosaicado de tiles
rasterio	Lectura de GeoTIFF y cálculo de CHM
geopandas	Exportar hull como GeoJSON
matplotlib	Visualización de perfiles y rasters

Referencias

- PDAL Documentation
- PNOA-LiDAR (IGN)
- Especificaciones técnicas PNOA 3ª cobertura
- Estándar LAS (ASPRS)
- Sistema de referencia ETRS89 (IGN)
- Geoide EGM08-REDNAP (IGN)